

API Testing with Postman Lab Manual

1.1 Overview of APIs, API Testing and Postman

An API allows one software application to communicate with another. In API testing, requests are sent to an endpoint and the responses are checked to confirm that the API behaves as expected.

Postman is a tool used to create requests, send them to APIs, inspect responses, organize requests, add variables, write scripts, and automate testing.

API testing commonly checks:

- status codes
- response time
- headers
- response body
- returned values
- authentication behavior
- error handling
- Postman provides a user interface where requests can be created using methods such as GET, POST, PUT, and DELETE. It also allows the user to add query parameters, headers, authorization details, body content, scripts, and tests.

1.2 Exploring Basic Postman Features and Making a Simple API Request

Step 1: Open Postman

Launch the Postman desktop application or open Postman in the browser.

Step 2: Create a New Request

Click New and select HTTP Request.

Step 3: Select the Request Method

From the method dropdown, choose:

GET

Step 4: Enter the Request URL

Type the following URL into the request field:

`https://postman-echo.com/get`

Step 5: Send the Request

Click the Send button.

Step 6: Inspect the Response

After sending the request, review the response section:

- status code
- response body
- response headers
- response time
- response size

Step 7: Add Query Parameters

Use the request URL:

<https://postman-echo.com/get?course=api-testing&tool=postman>

Or enter the same values in the Params tab:

- course = api-testing
- tool = postman

Click Send again and inspect the response body. The returned JSON should include the values sent in the request.

Step 8: Create a POST Request

Create another request and change the method to:

POST

Use the URL:

<https://postman-echo.com/post>

Step 9: Add a JSON Request Body

Open the Body tab, select raw, then choose JSON.

Enter the following:

```
{
  "name": "Student One",
  "course": "API Testing",
  "level": "Beginner"
}
```

Step 10: Send the POST Request

Click Send and inspect the response. The response should echo the submitted JSON.

1.3 Exploring Advanced Postman Features: Collections and Environments

A **collection** is a group of saved API requests in Postman that helps organize, manage, and reuse related requests in one place.

An **environment** is a set of variables that allows you to switch between different configurations (such as development, testing, or production) without changing the request manually.

Step 1: Create a Collection

Click New and choose Collection.

Enter the collection name:

API Testing Lab Collection

Add a description if required, then save the collection.

Step 2: Save Requests to the Collection

Save the GET and POST requests created earlier into the collection.

Step 3: Create an Environment

Open the Environments section and create a new environment named:

Lab Environment

Step 4: Add Environment Variables

Add the following variables:

Variable Name	Value
baseUrl	https://postman-echo.com
courseName	API Testing

Save the environment.

Step 5: Select the Environment

Choose Lab Environment from the environment dropdown.

Step 6: Use the Environment Variable in a Request

Update the GET request URL to:

{{baseUrl}}/get

Click Send and verify that the request works correctly.

1.4 Exploring Advanced Postman Features: Variable Types

A **variable** in Postman is a named value that can be stored and reused across requests, collections, environments, and scripts. Variables help make requests dynamic and flexible by avoiding hardcoding values.

Variable types (or scopes) define where a variable can be accessed and how widely it can be used. Different variable types allow better organization and control over data used in API testing.

Global Level Variables

Global variables are available broadly across the workspace unless a narrower scope overrides them.

Step 1

Open variable management in Postman.

Step 2

Create the variable:

```
username = global_user
```

Step 3

Save the variable.

Collection Level Variables

Collection variables are available to all requests inside a specific collection.

Step 1

Open the collection settings.

Step 2

Add the variable:

```
username = collection_user
```

Step 3

Save the collection.

Environment Level Variables

Environment variables are used within the currently selected environment.

Step 1

Open the Lab Environment.

Step 2

Add the variable:

```
username = env_user
```

Step 3

Save the environment.

Local Variables

Local variables are created temporarily during request execution and exist only within that request.

Step 1

Open a request.

Step 2

Go to the Pre-request Script tab.

Step 3

Add:

```
pm.variables.set("username", "local_user");
```

Step 4

Save the request.

Data Variables

Data variables are values loaded from an external file when running a collection, allowing multiple test inputs.

Step 1

Create a file named:

users.json

Step 2

Add the following content:

```
[  
  {  
    "username": "data_user_1"  
  },  
  {  
    "username": "data_user_2"  
  }  
]
```

Step 3

Save the file for later use in the Collection Runner or Newman.

Step 4: Use a Variable in a Request

Create or update a GET request:

```
{{baseUrl}}/get?user={{username}}
```

Step 5: Send the Request

Run the request and observe which value is used for username.

1.5 Scoping and Priority of Variables

A **test** in Postman is a piece of JavaScript code used to verify that an API response behaves as expected.

An **assertion** is a condition that checks whether the actual result matches the expected result. If the condition is true, the test passes; if false, the test fails.

Postman uses the **Chai assertion library**, which provides different ways to write assertions.

Types of Assertions in Postman

1. **Status Code Assertions**

Used to verify the HTTP status code returned by the API.

Example: checking if response is 200 (OK)

2. **Response Time Assertions**

Used to check how fast the API responds.

3. **Equality Assertions**

Used to compare actual values with expected values.

4. **Boolean Assertions**

Used to verify true/false conditions.

5. **Existence Assertions**

Used to check whether a field or value exists in the response.

6. **Type Assertions**

Used to verify the data type (string, number, object, etc.).

Step 1: Create a Request for Testing

Use the request:

```
{{baseUrl}}/get?course=api
```

Step 2: Open the Post-response Script Section

Go to the Scripts area and select Post-response.

A **post-response script** runs after the API response is received and is used to write tests.

Step 3: Add a Status Code Test

Enter:

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

This is a **status code assertion**.

Step 4: Add a Response Time Test

Add:

```
pm.test("Response time is less than 1000ms", function () {  
    pm.expect(pm.response.responseTime).to.be.below(1000);  
});
```

This is a **response time assertion**.

Step 5: Add a Response Body Assertion

Add:

```
pm.test("Response contains course parameter", function () {  
    const jsonData = pm.response.json();  
    pm.expect(jsonData.args.course).to.eql("api");  
});
```

This is an **equality assertion** (checking exact match).

Step 6: Send the Request

Click Send.

Step 7: Check the Test Results

View the test results section and confirm that all tests pass.

Step 8: Observe a Failed Assertion

Modify the expected value:

```
pm.test("Course parameter should be testing", function () {  
    const jsonData = pm.response.json();  
    pm.expect(jsonData.args.course).to.eql("testing");  
});
```

Send the request again and observe the failed test result.

1.7 Pre-request and Post-response Script Features

A **script** in Postman is a piece of JavaScript code used to automate tasks such as setting variables, modifying requests, or validating responses.

A **pre-request script** runs **before** a request is sent. It is used to prepare dynamic data such as timestamps, tokens, or random values.

A **post-response script** (also called a test script) runs **after** a response is received. It is used to process the response, extract values, and store data for use in other requests.

A **dynamic value** is a value that changes each time a request is executed, such as a timestamp or random number.

Step 1: Create a Request with a Dynamic Timestamp

Use the request:

```
{{baseUrl}}/get?timestamp={{timestamp}}
```

Step 2: Add a Pre-request Script

In the Pre-request Script tab, add:

```
pm.variables.set("timestamp", new Date().toISOString());
```

A **timestamp** is a date and time value recorded in a standard format.

Step 3: Send the Request

Click Send and inspect the returned timestamp value.

Step 4: Generate a Random Value

Use:

```
{{baseUrl}}/get?randomId={{randomId}}
```

In the Pre-request Script tab, add:

```
pm.variables.set("randomId", Math.floor(Math.random() * 10000));
```

A **random value** is a number generated automatically to create unique data for each request.

Step 5: Send the Request Multiple Times

Observe that the value changes each time.

Step 6: Save Response Data into an Environment Variable

Use the request:

```
{{baseUrl}}/get?student=Evangadi
```

In the Post-response Script tab, add:

```
const jsonData = pm.response.json();  
pm.environment.set("savedStudent", jsonData.args.student);
```

An **environment variable** stores data that can be reused across multiple requests within a selected environment.

Step 7: Send the Request

Click Send.

Step 8: Reuse the Saved Variable

Create another request:

```
{{baseUrl}}/get?copied={{savedStudent}}
```

Step 9: Send the Request

Confirm that the saved value appears in the new request.

Step 10: Send an Additional Request from a Script

In a script tab, add:

```
pm.sendRequest("https://postman-echo.com/get?source=script", function (err, res) {  
    console.log(res.json());  
});
```

The function **pm.sendRequest()** allows sending additional HTTP requests from within a script.

Step 11: Run the Request

Open the Postman Console if needed and inspect the logged response.

The **Postman Console** is a debugging tool used to view request details, responses, and script output.

1.8 Introduction to Mock Servers in Postman

In Postman, a **Mock Server** is a feature that allows you to simulate an API by returning predefined responses without calling a real backend service.

A **Mock Example** is a saved response inside a request that Postman uses to generate responses when the mock server receives a request.

In the Postman API tool, mock servers are directly created from **collections and saved examples**, making it easy to test APIs during development.

Why Use Mock Servers in Postman

- Allows testing APIs without a real backend
- Enables frontend and backend teams to work independently

- Provides consistent and predictable responses
- Helps simulate different scenarios (success, error, edge cases)
- Reduces dependency on external systems

Steps to Create and Use a Mock Server in Postman

Step 1: Create a Request in Postman

Create a new request inside your collection:

GET {{baseUrl}}/get?mock=true

Step 2: Send the Request

Click **Send** to get a real response from the API.

Step 3: Save the Request to a Collection

Click **Save** and store the request in your collection (e.g., *API Testing Lab Collection*).

Step 4: Save Response as an Example

In the response section:

- Click **Save Response**
- Select **Save as Example**
- Name it:

Mock Success Example



This example will be used by the mock server.

Step 5: Create a Mock Server

In Postman:

- Open the collection or request
- Click **...** (**More options**)
- Select **Mock Collection** or **Create Mock Server**

Step 6: Configure the Mock Server

- Select the saved example
- Choose environment if needed
- Click **Create Mock Server**

Postman will generate a **mock server URL**.

Step 7: Send Request to Mock Server

Replace your original URL with the mock URL:

https://<mock-id>.mock.pstmn.io/get?mock=true

Step 8: Verify Mock Response

Click **Send** and confirm:

- Response matches the saved example
- No real API call is made

1.9 Introduction to Automated Testing Using Newman

Newman is Postman's command-line collection runner. It allows Postman collections to be executed outside the Postman interface. This is useful for terminal-based testing, continuous integration pipelines, scheduled execution, and automated reporting.

With Newman, a user can:

- run a collection from the command line
- include an environment file
- include data files
- generate reports
- integrate Postman tests into automation pipelines

1.10 Setting Up Newman and Running Collections with Newman

Step 1: Install Node.js

Make sure Node.js is installed on the system.

Step 2: Install Newman

Open the terminal or command prompt and run:

```
npm install -g newman
```

Step 3: Verify the Installation

Run:

```
newman -v
```

Step 4: Export the Collection from Postman

Export the collection and save it as:

```
api-testing-lab.postman_collection.json
```

Step 5: Export the Environment

Export the environment and save it as:

```
lab-environment.postman_environment.json
```

Step 6: Run the Collection

In the terminal, run:

```
newman run api-testing-lab.postman_collection.json
```

Step 7: Run the Collection with an Environment

Run:

```
newman run api-testing-lab.postman_collection.json -e lab-  
environment.postman_environment.json
```

Step 8: Create a Data File

Create a file named:

users.json

Add:

```
[  
  { "username": "alice" },  
  { "username": "bob" }  
]
```

Step 9: Run the Collection with Iteration Data

Use:

```
newman run api-testing-lab.postman_collection.json -e lab-  
environment.postman_environment.json -d users.json
```

Step 10: Generate Reports

Run:

```
newman run api-testing-lab.postman_collection.json -r cli,json,junit
```

Step 11: Review the Output

Inspect:

- request results
- passed tests
- failed tests
- summary details
- generated report files

Step 12: Troubleshoot if Newman Fails

Check the following:

- Node.js installation
- correct file paths
- valid exported JSON files
- correct collection structure
- supported script features

1.11 Introduction to Postman Monitors

- **Monitor** : A Postman Monitor automatically runs a collection at scheduled intervals to check if the API is working correctly over time.

- **Scheduling**: Scheduling defines how often the monitor runs (e.g., every 5 minutes, hourly, daily).

Steps to Create and Use a Postman Monitor

Step 1: Prepare a Collection with Tests

Make sure the collection contains at least one request with passing tests.

Example request:

GET {{baseUrl}}/get?health=ok

Example test:

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

Step 2: Open the Monitor Creation Option

From the collection options, choose to create a monitor.

Step 3: Select the Collection

Choose the collection you want to monitor.

Step 4: Attach an Environment if Needed

Select the correct environment if the collection uses variables.

Step 5: Configure the Schedule

Choose how often the monitor should run.

Step 6: Save the Monitor

Complete the monitor setup.

Step 7: Run or Wait for the Scheduled Execution

Start the monitor manually if available, or wait for the scheduled run.

Step 8: View Monitor Results

Inspect:

- run status
- passed and failed tests
- failed requests
- timestamps
- response behavior over time

Step 9: Interpret the Results

Use the monitor results to determine whether the API continues to behave correctly across repeated scheduled runs.